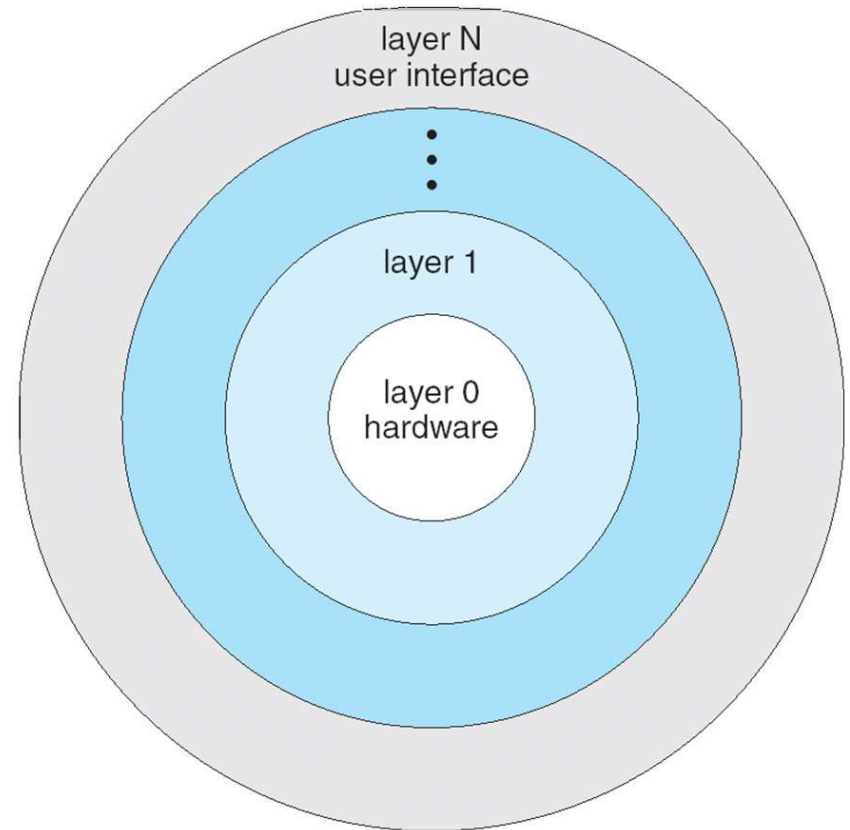


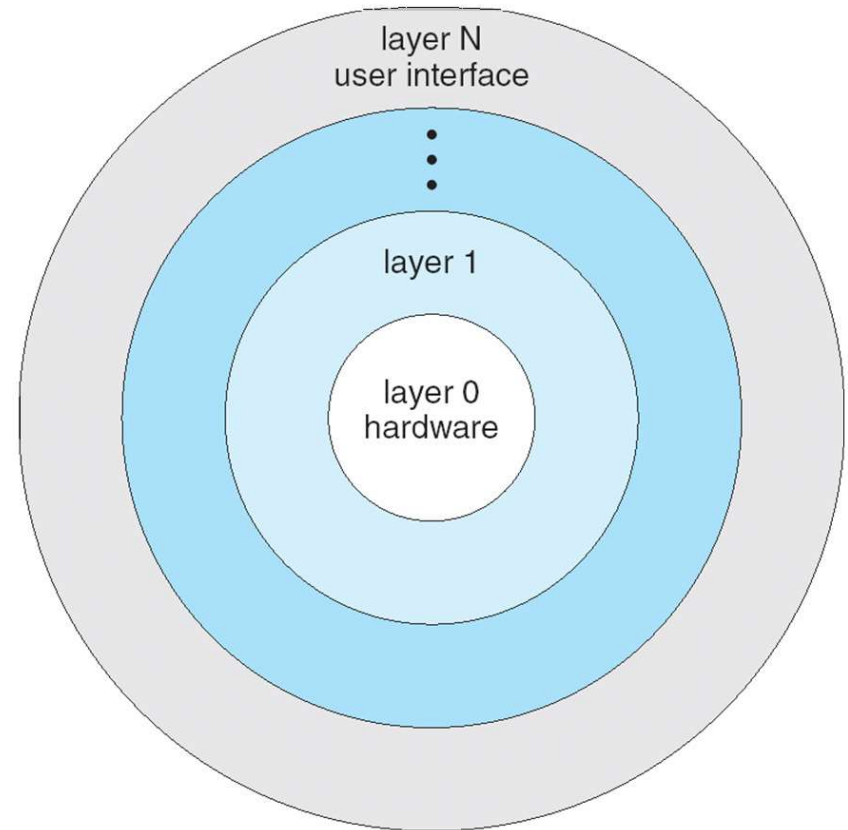
Layered Approach

- The operating system is divided
 - into a number of layers (levels),
 - each built on top of lower layers.
 - **The bottom layer (layer 0), is the hardware;**
 - **The highest (layer N) is the user interface.**



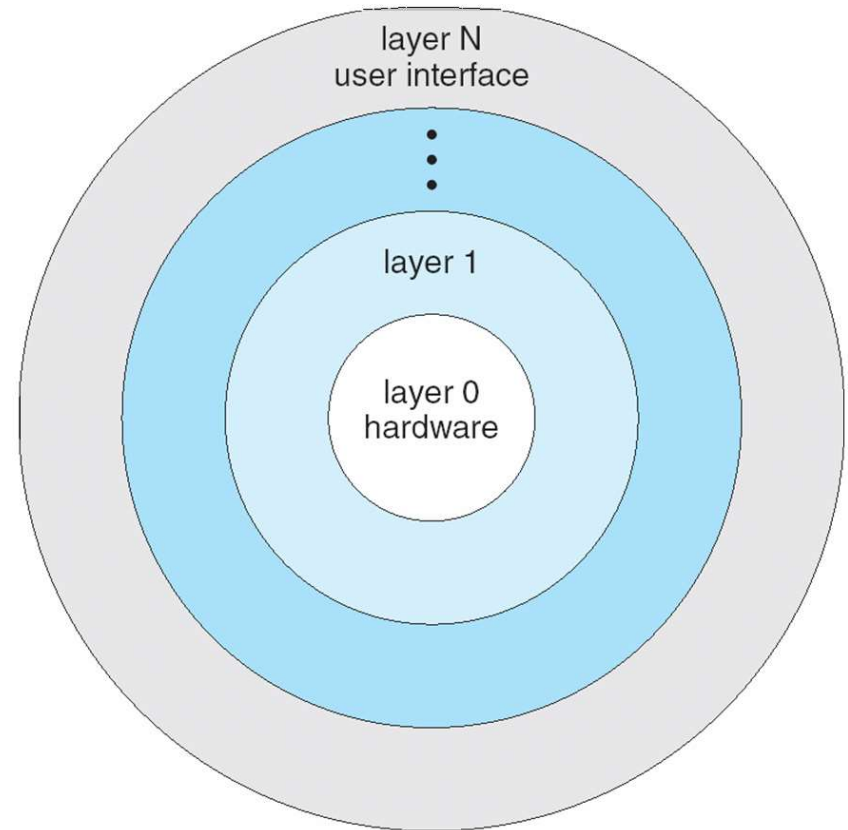
Layered Approach

- An OS layer say M,
 - **consists of data structures and**
 - **a set of routines**
 - **that can be invoked by higher layers.**
 - **Layer M can invoke operations on lower layers.**
- With modularity,
 - layers are selected such that
 - **each uses functions and services of only lower-level layers**



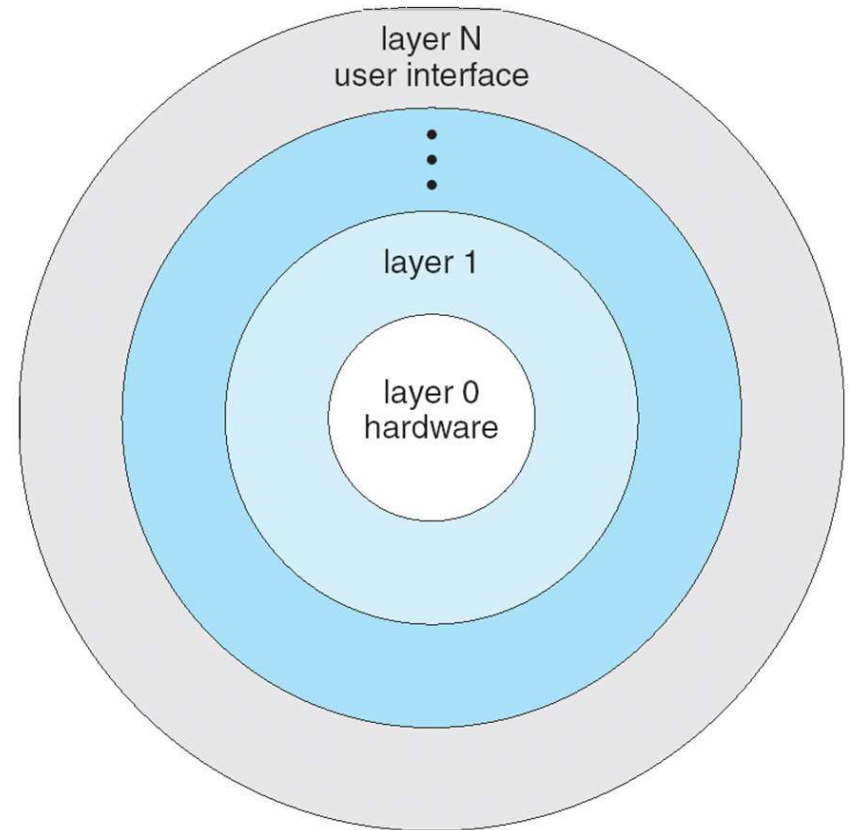
Layered Approach

- It simplifies debugging and system verification.
- **The 1st layer can be debugged**
 - **without any concern for the rest of the system.**
- Once the 1st layer is debugged,
 - its correct functioning can be assumed while
 - the second layer is being debugged and so on.



Layered Approach

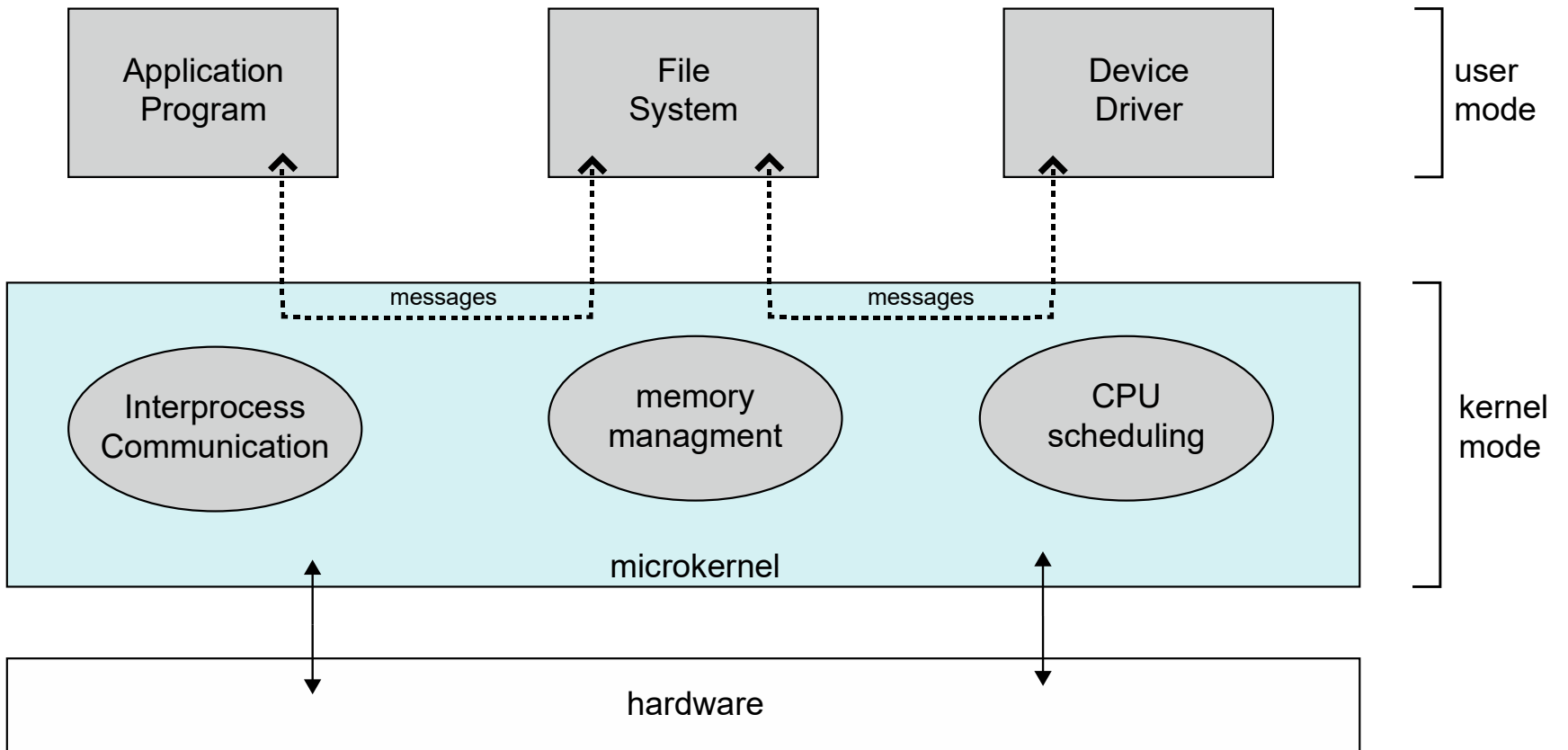
- If error arises during debugging of a particular layer,
 - **the error must be on that layer,**
 - because the **layers below it are already debugged.**



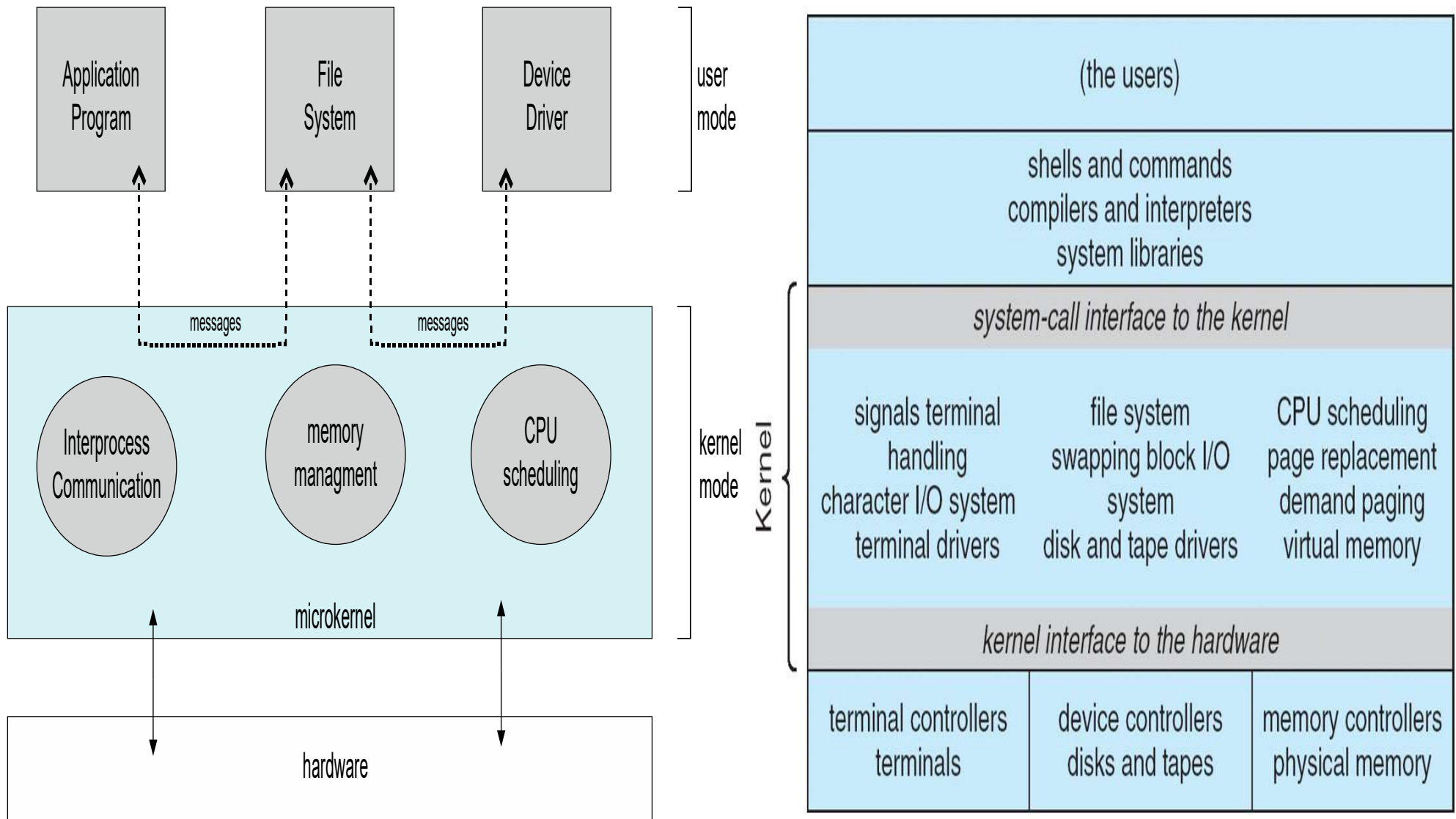
Microkernel System Structure

- Moves as much from the kernel into user space
- **Mach** example of **microkernel**
 - Mac OS X kernel (**Darwin**) partly based on Mach
- Communication takes place between user modules using **message passing**

Microkernel System Structure



Comparison-Microkernel System UNIX Kernel



Microkernel System Structure

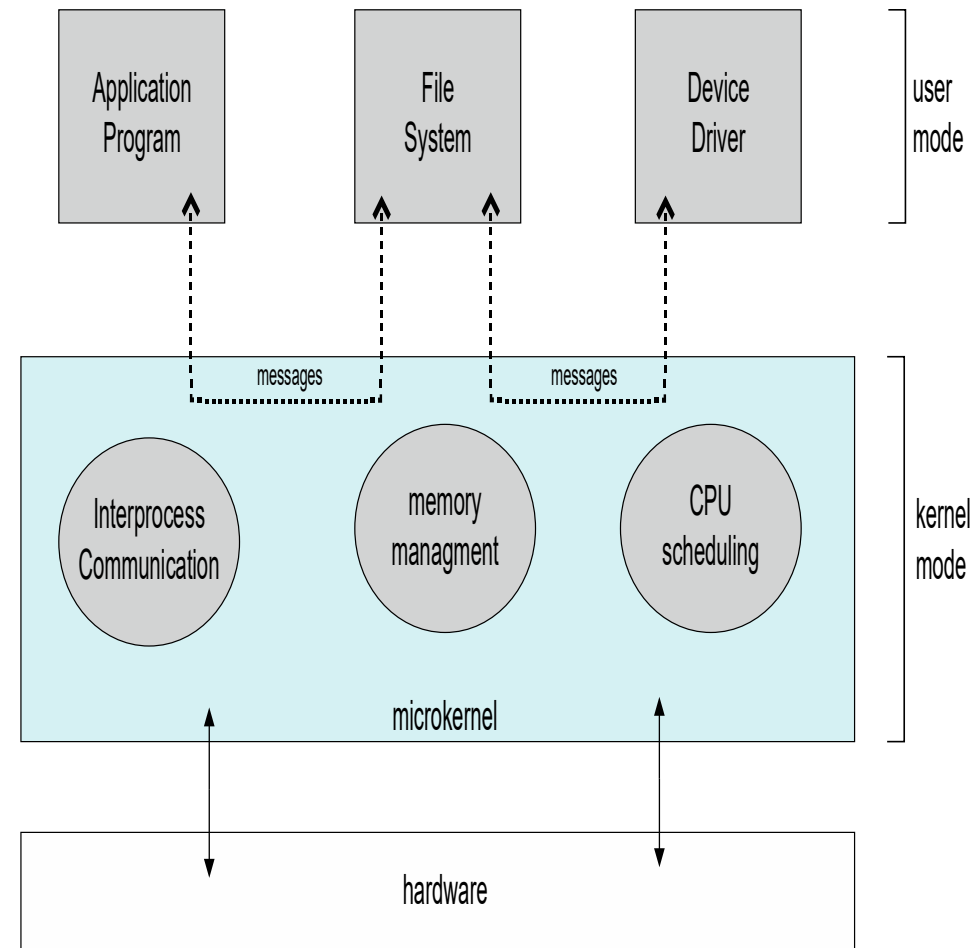
How?

- **Remove all non essential components from the kernel and**
 - implementing them as system and user level programs.
- **Result=Smaller=Micro Kernel**

Microkernel System Structure

How?

- Microkernels provide
 - minimal process and memory management, communication facility.
- Main fn
 - **Provide a communication facility**
 - between client program and various services that are also running in user space,
 - **Communication provided by Message passing**



Microkernel System Structure

When ? Where?

- As the UNIX OS expanded, kernel became large and difficult to manage.
- In mid **1980s**, researchers at **Carnegie Mellon University** developed an OS called **Mach**,
 - that modularizes the kernel using **The Microkernel approach**.

Microkernel System Structure

- Benefits:
 - **Easier to extend** a microkernel
 - **All new services added to user space , no modification of the kernel**
 - **Easier to port** the operating system to new architectures
 - **As no modifications to kernel, changes tend to be fewer, a smaller kernel**

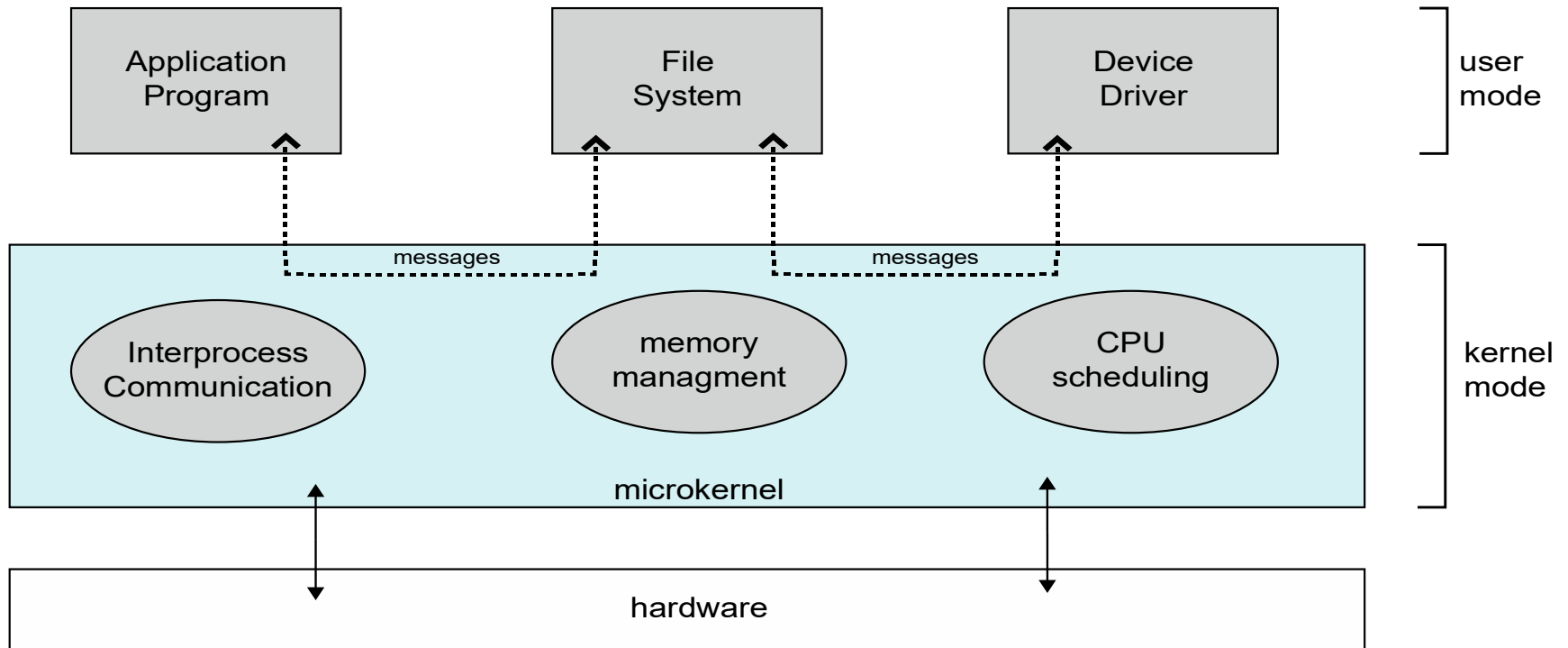
Microkernel System Structure

- Benefits:
 - More reliable (less code is running in kernel mode) and More secure
 - **More services running as user rather than kernel,**

- **If a service fails, the rest of the operating system remains untouched.**

Microkernel System Structure

- Detriments:
 - **Performance overhead** of user space to kernel space communication



Linux Kernel

- Think about it like this.
- The kernel is a busy personal assistant for a powerful executive (the hardware).
- It's the assistant's job :-
 - to relay messages and requests (processes) from employees and the public (users) to the executive,
 - to remember what is stored where (memory), and
 - to determine who has access to the executive at any given time and for how long.

Courtesy: <https://www.redhat.com/en/topics/linux/what-is-the-linux-kernel>

Linux Kernel

- Linux Kernel a busy personal assistant that
 - manages all system resources and
 - that interacts directly with the computer hardware.
- The Linux[®] kernel is
 - **the main component of a Linux operating system (OS) and**
 - **is the core interface between a computer's hardware and its processes.**
 - It communicates between the two, managing resources as efficiently as possible

Linux Kernel

- The kernel is so named because—
 - **like a seed inside a hard shell**
 - it exists within the OS and
 - controls **all the major functions** of the hardware



Linux Kernel

- We need much more than this kernel
 - to produce a full operating system.
- The Linux kernel
 - **forms the core of the Linux project, but**
 - **other components make up the complete Linux operating system**

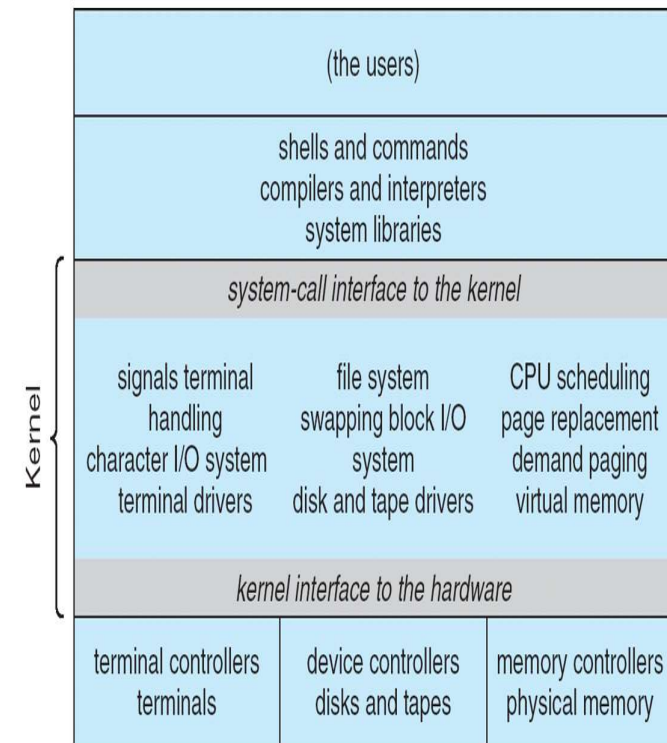
Linux Kernel

- **Linux kernel** is composed
 - **entirely of code written from scratch** specifically for the Linux project,
- Whereas the **Linux system** includes
 - a multitude of components some written from **scratch**
 - **other borrowed** from other development projects and
 - others created in **collaboration with other teams.**

Linux Kernel

- The kernel, if implemented properly,
 - **is invisible to the user,**
 - **working in its own little world known as kernel space,**
 - where it allocates memory and
 - keeps track of where everything is stored.
- **What the user sees—like web browsers and files—are known as the user space.**
- **These applications interact with the kernel**
 - **through a system call interface (SCI).**

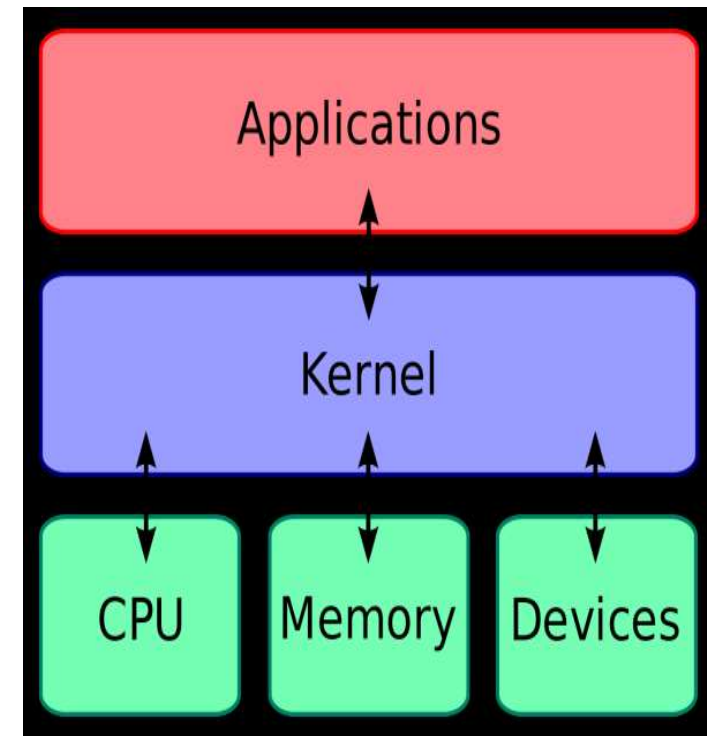
Courtesy: <https://www.redhat.com/en/topics/linux/what-is-the-linux-kernel>



Where the kernel fits within the OS?

To put the kernel in context, you can think of a Linux machine as having 3 layers:

- The hardware
- The Linux kernel: The core of the OS. (See? It's right in the middle.) It's software residing in memory that tells the CPU what to do.
- User processes: These are the running programs that the kernel manages.
 - User processes are what collectively make up user space.
 - The kernel also allows these processes and servers to communicate with each other (known as inter-process communication, or IPC).

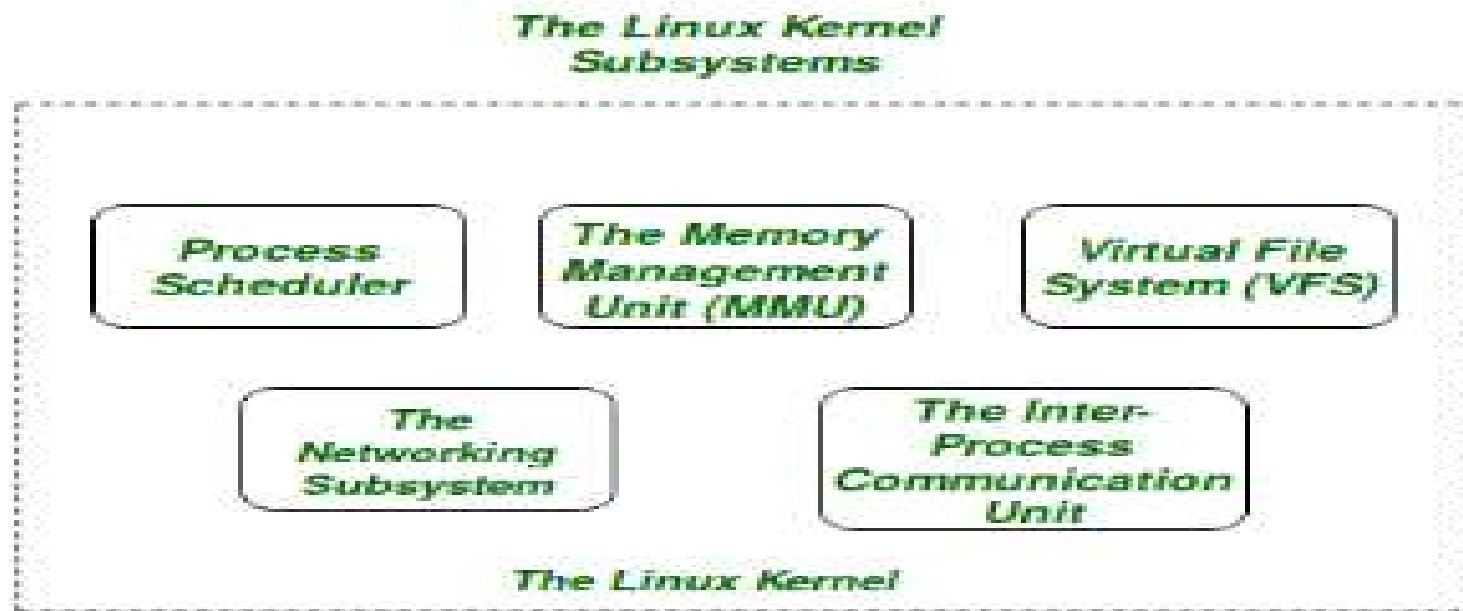


Courtesy: <https://www.redhat.com/en/topics/linux/what-is-the-linux-kernel>

Linux Kernel

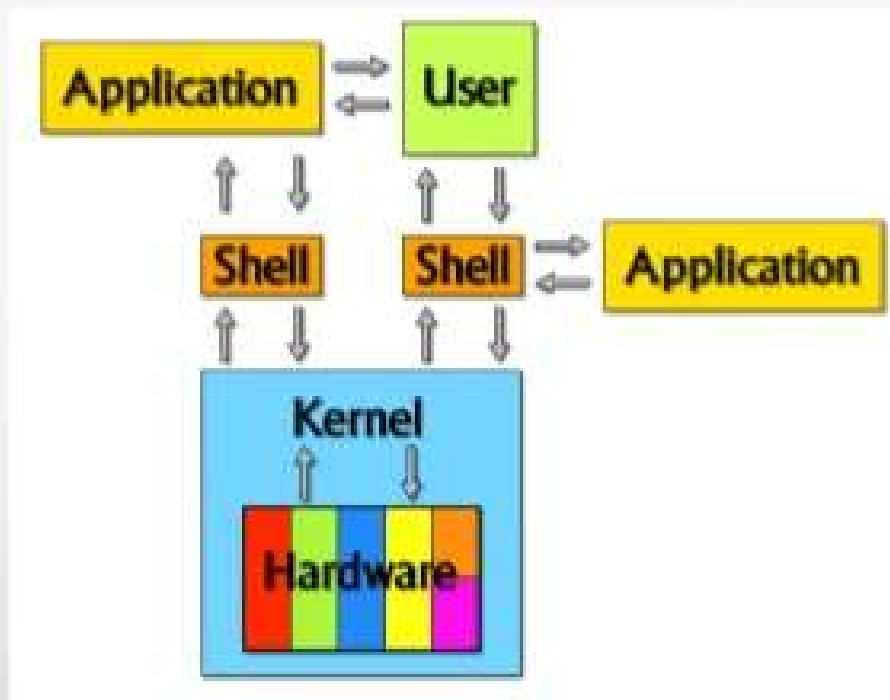
The **Core Subsystems** of the **Linux Kernel** are as follows:

- The Process Scheduler/ Process management
- The Memory Management Unit (MMU)
- The Virtual File System (VFS) /File management
- The Networking Unit
- Inter-Process Communication Unit
- Device management/ I/O management



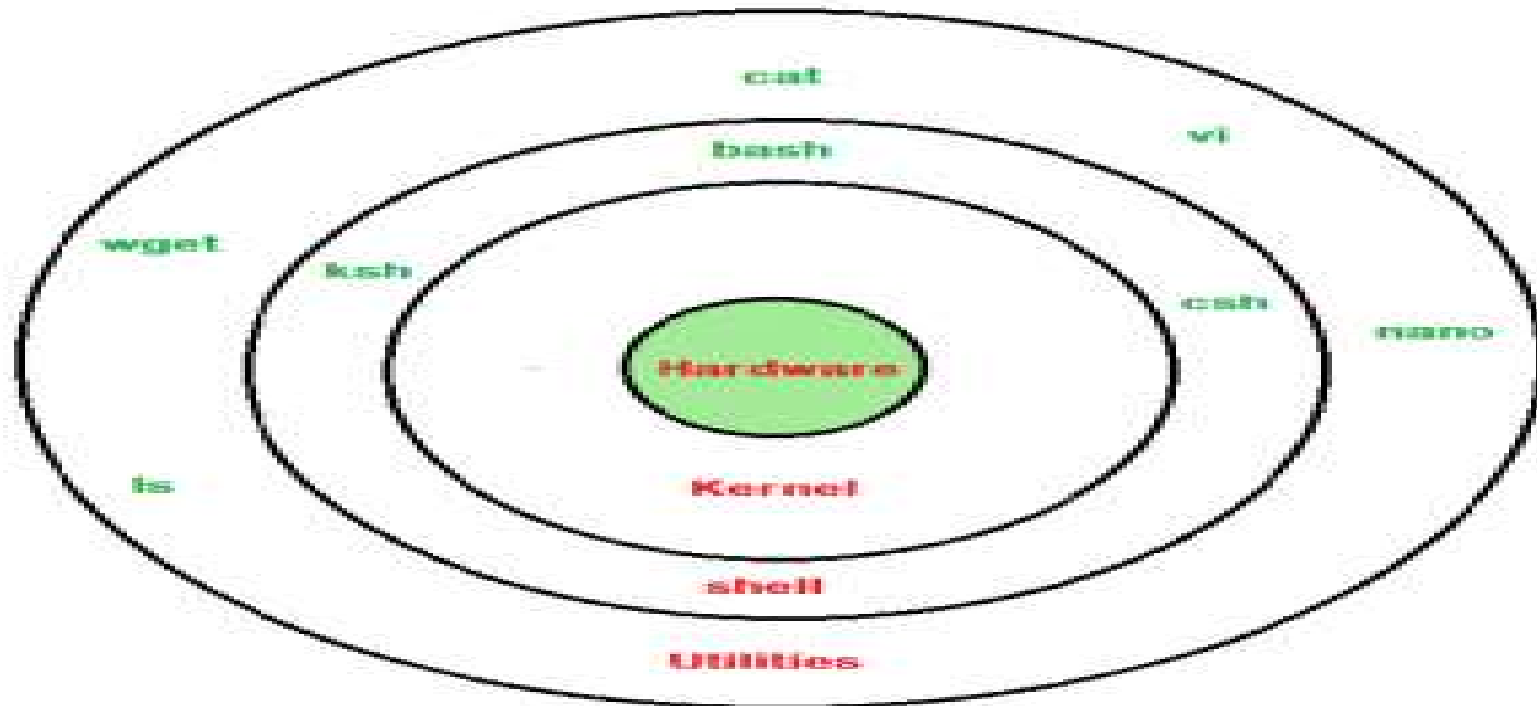
What Is Kernel?

- central module of an operating system (OS)
- the part of the operating system that loads first, and it remains in main memory.
- can be contrasted with a shell, the outermost part of an operating system that interacts with user commands.
- Manage your hardware and system resources



Linux Kernel

- It is often mistaken that **Linus Torvalds** has developed **Linux OS**,
 - but actually he is only responsible for development of Linux kernel.
- **Complete Linux system = Kernel + GNU system utilities and libraries + other management scripts + installation scripts.**



What is Shell?

- A shell is special user program which provide an interface to user to use operating system services.
 - Shell accept human readable commands from user and convert them into something which kernel can understand.

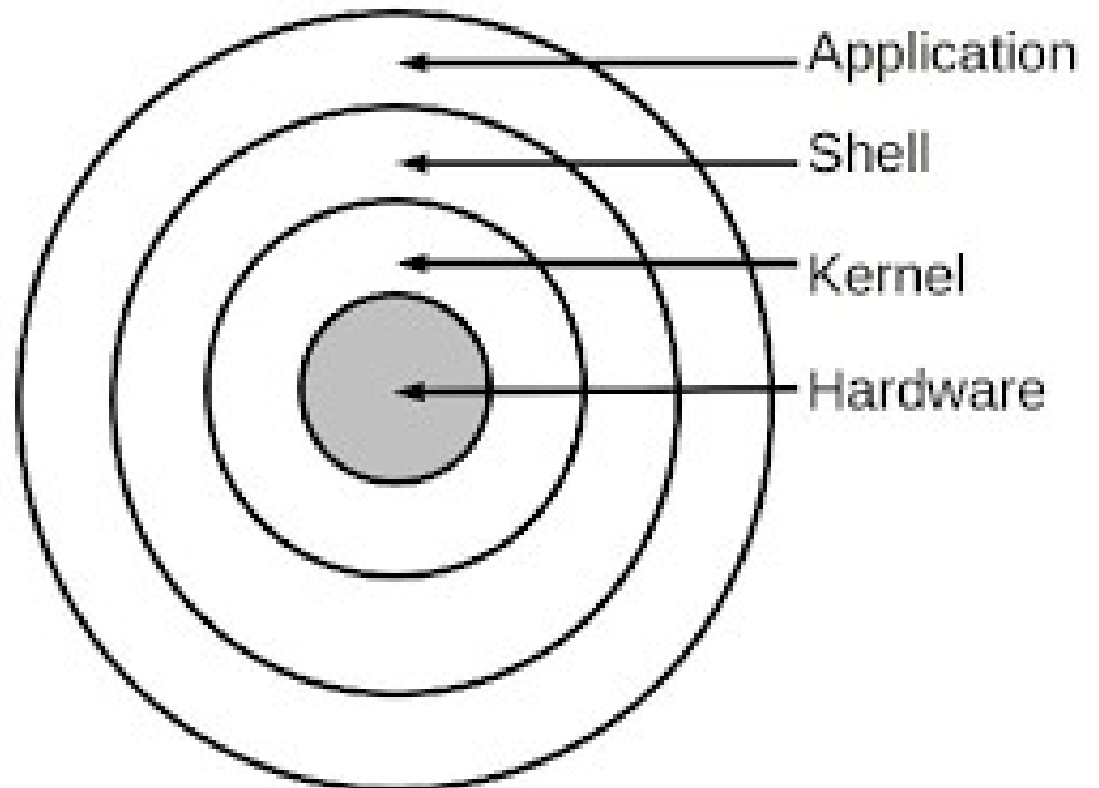
What is Shell?

- It is a **command language interpreter** that execute commands read from input devices such as keyboards or from files.

- **The shell gets started when the user logs in or start the terminal.**

What is Shell?

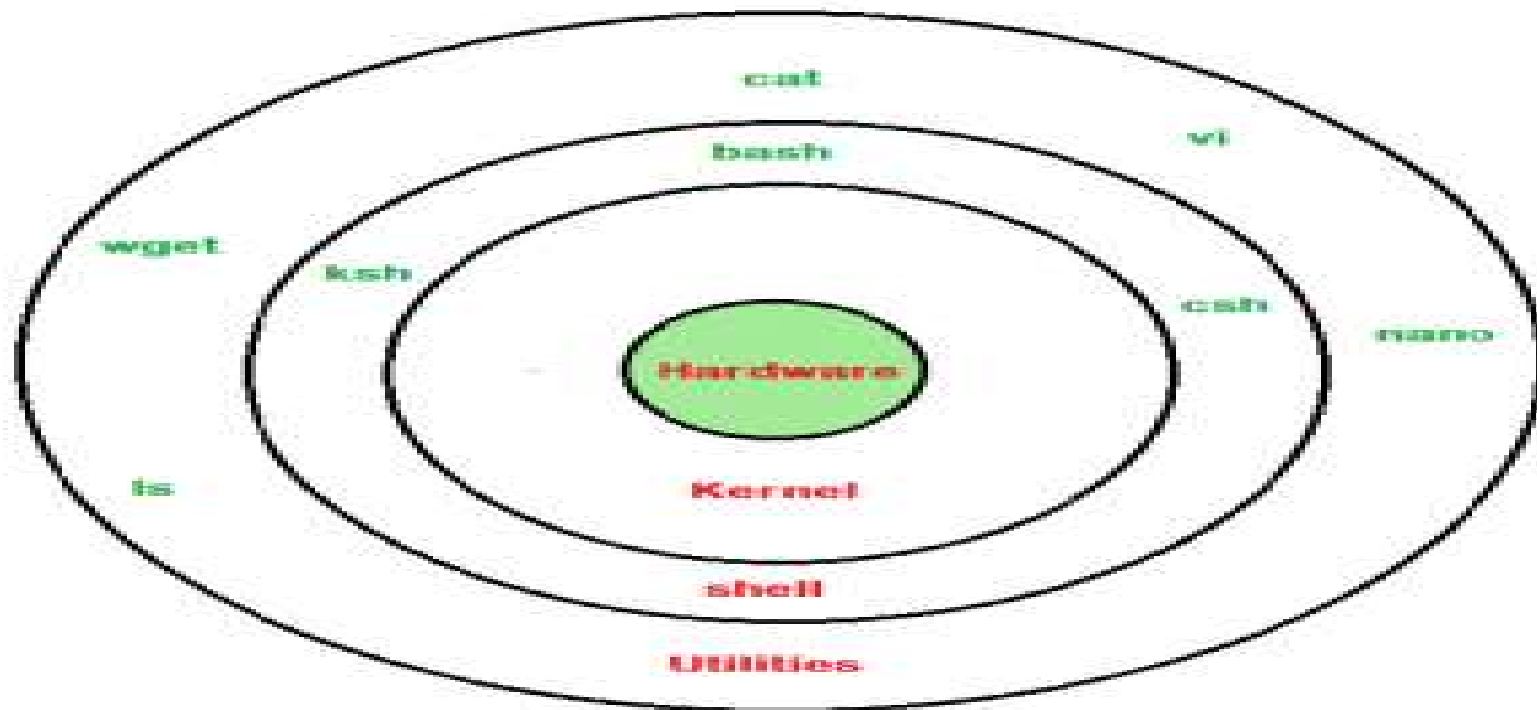
- It is named a shell because it is the outermost layer around the operating system.



What is Shell?

Shell is broadly classified into two categories-

- Command Line Shell
- Graphical shell

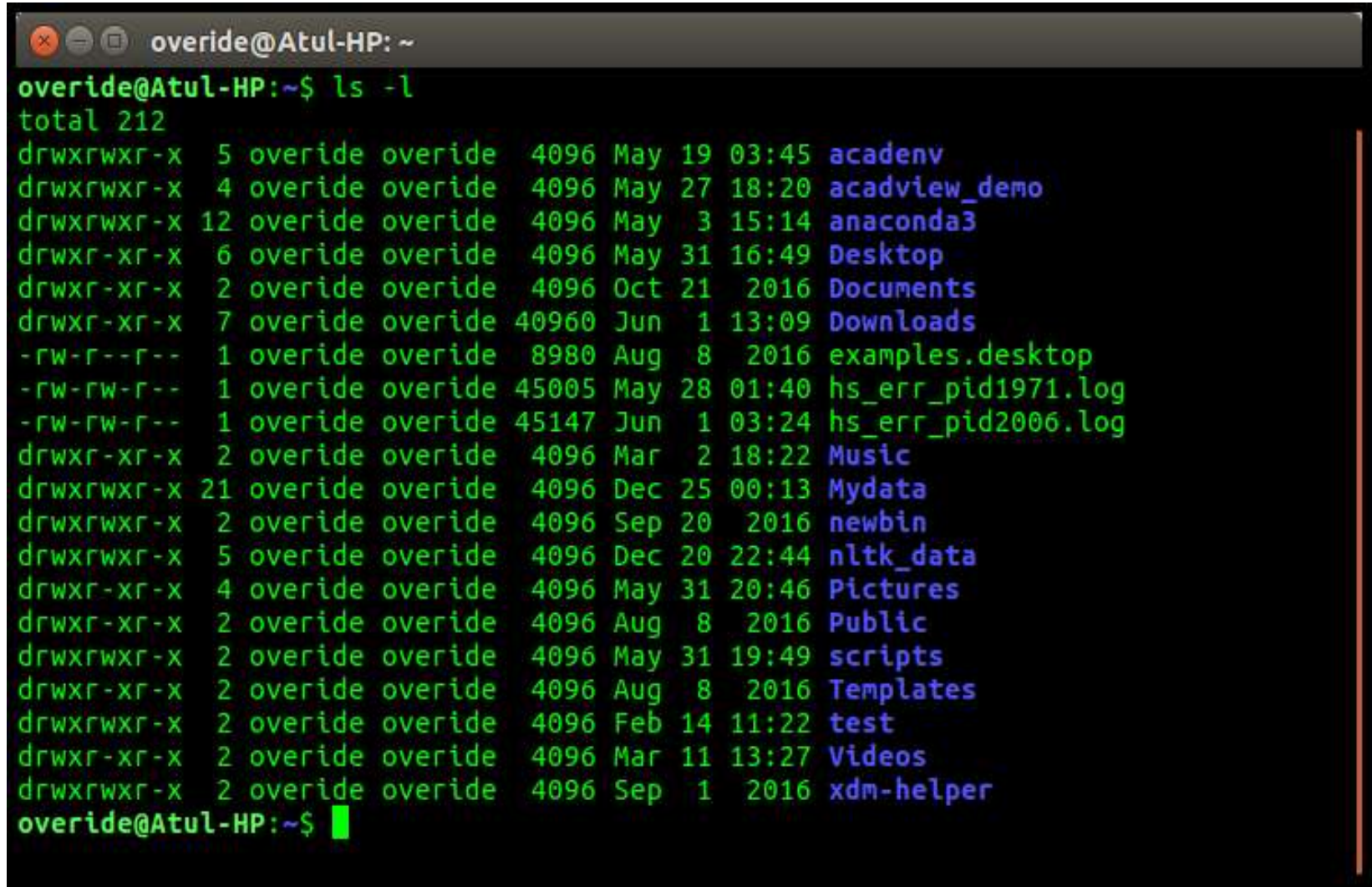


Command Line Shell

- Shell can be accessed by user using a command line interface.
- A special program called
 - Terminal in linux/macOS or
 - Command Prompt in Windows OS
 - is provided to type in the human readable commands such as “cat”, “ls” etc. for execution. The result is then displayed on the terminal to the user.

Command Line Shell

- A terminal in Ubuntu 16.4 system looks like this –

A terminal window titled 'override@Atul-HP: ~' showing the output of the 'ls -l' command. The output lists various files and directories with their permissions, owner, group, size, date, and name. The files are: acadenv, acadview_demo, anaconda3, Desktop, Documents, Downloads, examples.desktop, hs_err_pid1971.log, hs_err_pid2006.log, Music, Mydata, newbin, nltk_data, Pictures, Public, scripts, Templates, test, Videos, and xdm-helper.

```
override@Atul-HP:~$ ls -l
total 212
drwxrwxr-x  5 override override 4096 May 19 03:45 acadenv
drwxrwxr-x  4 override override 4096 May 27 18:20 acadview_demo
drwxrwxr-x 12 override override 4096 May  3 15:14 anaconda3
drwxr-xr-x  6 override override 4096 May 31 16:49 Desktop
drwxr-xr-x  2 override override 4096 Oct 21  2016 Documents
drwxr-xr-x  7 override override 40960 Jun  1 13:09 Downloads
-rw-r--r--  1 override override 8980 Aug  8  2016 examples.desktop
-rw-rw-r--  1 override override 45005 May 28 01:40 hs_err_pid1971.log
-rw-rw-r--  1 override override 45147 Jun  1 03:24 hs_err_pid2006.log
drwxr-xr-x  2 override override 4096 Mar  2 18:22 Music
drwxrwxr-x 21 override override 4096 Dec 25 00:13 Mydata
drwxrwxr-x  2 override override 4096 Sep 20  2016 newbin
drwxrwxr-x  5 override override 4096 Dec 20 22:44 nltk_data
drwxr-xr-x  4 override override 4096 May 31 20:46 Pictures
drwxr-xr-x  2 override override 4096 Aug  8  2016 Public
drwxrwxr-x  2 override override 4096 May 31 19:49 scripts
drwxr-xr-x  2 override override 4096 Aug  8  2016 Templates
drwxrwxr-x  2 override override 4096 Feb 14 11:22 test
drwxr-xr-x  2 override override 4096 Mar 11 13:27 Videos
drwxrwxr-x  2 override override 4096 Sep  1  2016 xdm-helper
override@Atul-HP:~$
```

Command Line Shell

- Working with command line shell is bit difficult for the beginners because it's hard to memorize so many commands.
- It is very powerful, it allows user to store commands in a file and execute them together. This way any repetitive task can be easily automated.
- These files are usually called
 - Batch files in Windows and
 - Shell Scripts in Linux/macOS systems.

Graphical Shells

- Provide means for manipulating programs based on graphical user interface (GUI),
 - by allowing for operations such as opening, closing, moving and resizing windows, as well as switching focus between windows.
- **Window OS or Ubuntu OS can be considered as good example which provide GUI to user for interacting with program.**
- User do not need to type in command for every actions.

Graphical Shells

- Command-line shells require the user
- to be familiar with commands and
- their calling syntax, and
- to understand concepts about the shell-specific scripting language

- Graphical shells place
- a low burden on beginners and are easy to use.
- Since they also come with certain disadvantages, most GUI-enabled operating systems also provide CLI shells.

Shell Prompt

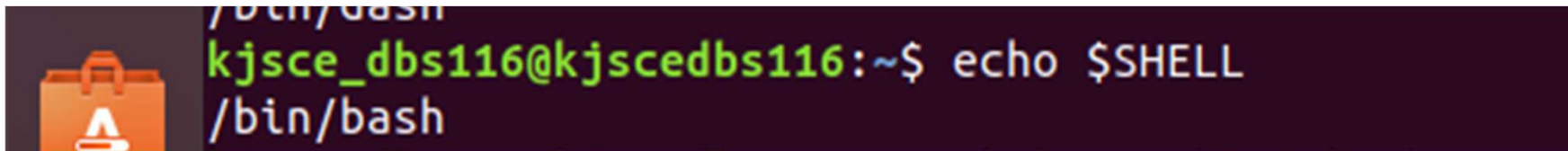
- The prompt, \$, which is called the **command prompt**, is issued by the shell.
- While the prompt is displayed, you can type a command.
- Shell reads your input after you press **Enter**.
- It determines the command you want to be executed by looking at the first word of your input. A word is an unbroken set of characters. Spaces and tabs separate words.

FINDING OUT YOUR SHELL

- When you are provided with a UNIX account, the system administrator chooses a shell for you.
 - To find out which shell was chosen for you, look at your prompt.
- 1) If you have a **\$** prompt, you're probably in a **Bash, Bourne** or a **Korn** shell.
 - 2) If you have a **%** prompt, you're probably in a **C shell**.

FINDING OUT YOUR SHELL

- Using the Echo command



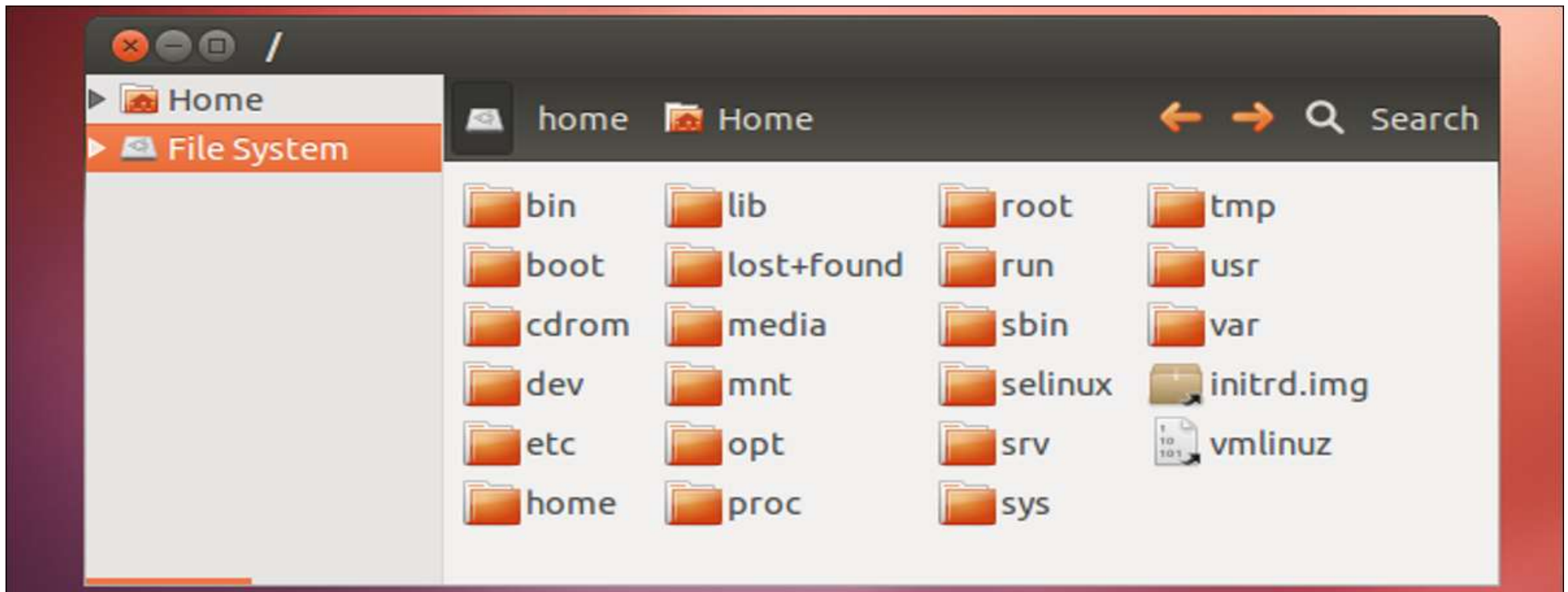
```
kjsce_dbs116@kjscedbs116:~$ echo $SHELL
/bin/bash
```

List all the shells on your PC?

- Using the Cat command

List all the shells on your PC?

- The list of all the shells which are currently installed in our Linux system is stored in the 'shells' file which is present in /etc folder of the system.
- It has read-only access by default and is modified automatically whenever we install a new shell in our system.

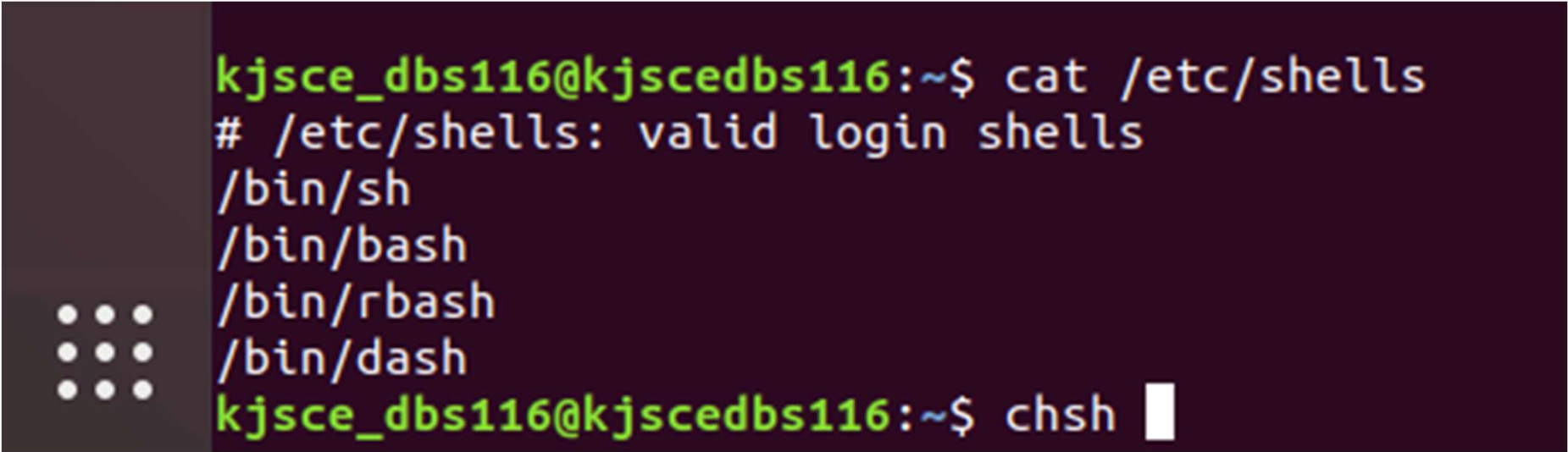


/etc — Configuration Files

- The /etc directory contains configuration files, which can generally be edited by hand in a text editor.
- Note that the /etc/ directory contains system-wide configuration files
- User-specific configuration files are located in each user's home directory.

List all the shells on your PC?

- The cat command displays the various installed shells along with their installation paths.

A terminal window with a dark purple background. On the left side, there is a vertical bar with a grid of nine white dots. The terminal text is as follows:

```
kjsce_dbs116@kjscedbs116:~$ cat /etc/shells
# /etc/shells: valid login shells
/bin/sh
/bin/bash
/bin/rbash
/bin/dash
kjsce_dbs116@kjscedbs116:~$ chsh
```

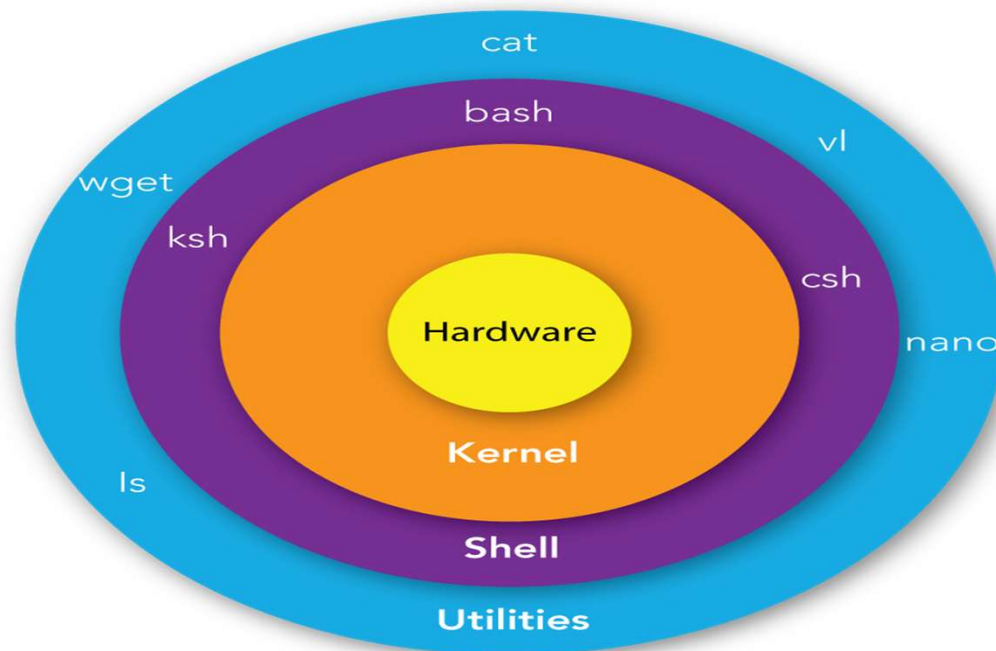
Change the shell

- **Using chsh Utility**
- chsh is the utility to change a user's login shell. chsh provides the -s option to change the user's shell.

```
sh-5.1$ grep nishant /etc/passwd
nishant:x:1000:1000::/home/nishant:/bin/sh
sh-5.1$ chsh -s /bin/bash nishant
Changing shell for nishant.
Password:
Shell changed.
sh-5.1$ grep nishant /etc/passwd
nishant:x:1000:1000::/home/nishant:/bin/bash
sh-5.1$
```


Shells

- **There are several shells** are available for Linux systems.
- Each shell does the same job but **understand different commands and provide different built in functions.**



Shell Types

- In UNIX there are two major types of shells:
 - The Bourne shell.
 - **If you are using a Bourne-type shell, the default prompt is the \$ character.**
 - The C shell.
 - **If you are using a C-type shell, the default prompt is the % character.**

<https://www.tutorialspoint.com/unix/unix-what-is-shell.htm>

The Bourne Shell

- The original UNIX shell written in the **mid-1970s by Stephen R. Bourne while he was at AT&T Bell Labs in New Jersey.**
- **Original UNIX shell-The Bourne shell was the first shell to appear on UNIX systems, thus it is referred to as "the shell".**
- Denoted as **sh**
- It is faster and more preferred.

<https://www.tutorialspoint.com/unix/unix-what-is-shell.htm>

The Bourne Shell

- It lacks features for interactive use like the ability to recall previous commands.
- It also lacks built-in arithmetic and logical expression handling.
- It is default shell for Solaris OS.

CSH (C SHell)

- The C Shell
- Denoted as csh
- Bill Joy created it at the University of California at Berkeley.
- It incorporated features such as aliases and command history.
- It includes helpful programming features like built-in arithmetic
- C-like expression syntax-shell syntax and usage are very similar to the C programming language.

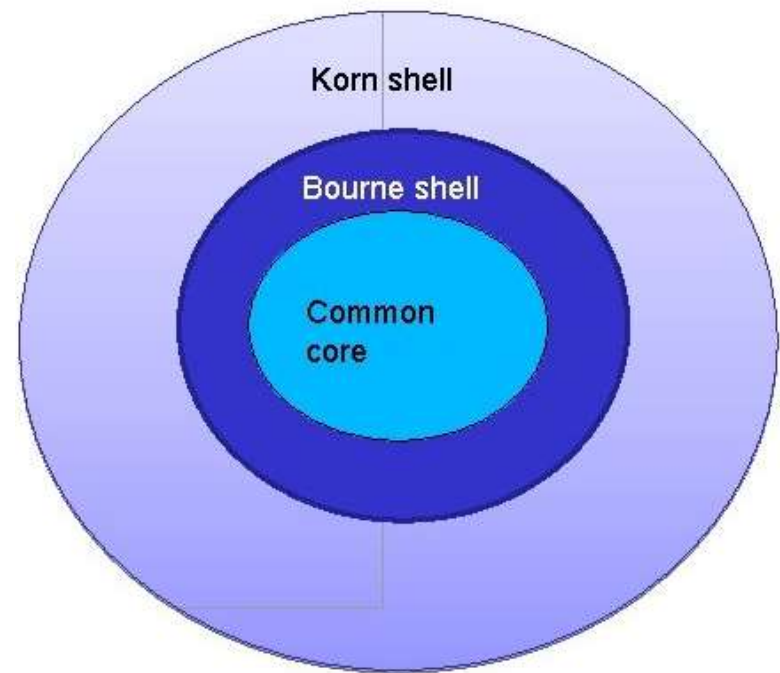
Shell Types

- There are again various subcategories for Bourne Shell which are listed as follows –
 - **Bourne shell (sh)**
 - **Korn shell (ksh)**
 - **Bourne Again shell (bash)**
 - **POSIX shell**
- The different C-type shells follow –
 - **C shell (csh)**
 - **TENEX/TOPS C shell (tcsh)**

<https://www.tutorialspoint.com/unix/unix-what-is-shell.htm>

The Korn Shell

- It is denoted as **ksh**
- It Was written by David Korn at AT&T Bell Labs.
- It is a superset of the Bourne shell. So it supports everything in the Bourne shell.



The Korn Shell

- It has interactive features.
- It includes features like built-in arithmetic and C-like arrays, functions, and string-manipulation facilities.
- It is faster than C shell. It is compatible with script written for C shell.
- The Korn Shell also was the base for the POSIX Shell standard specifications etc.

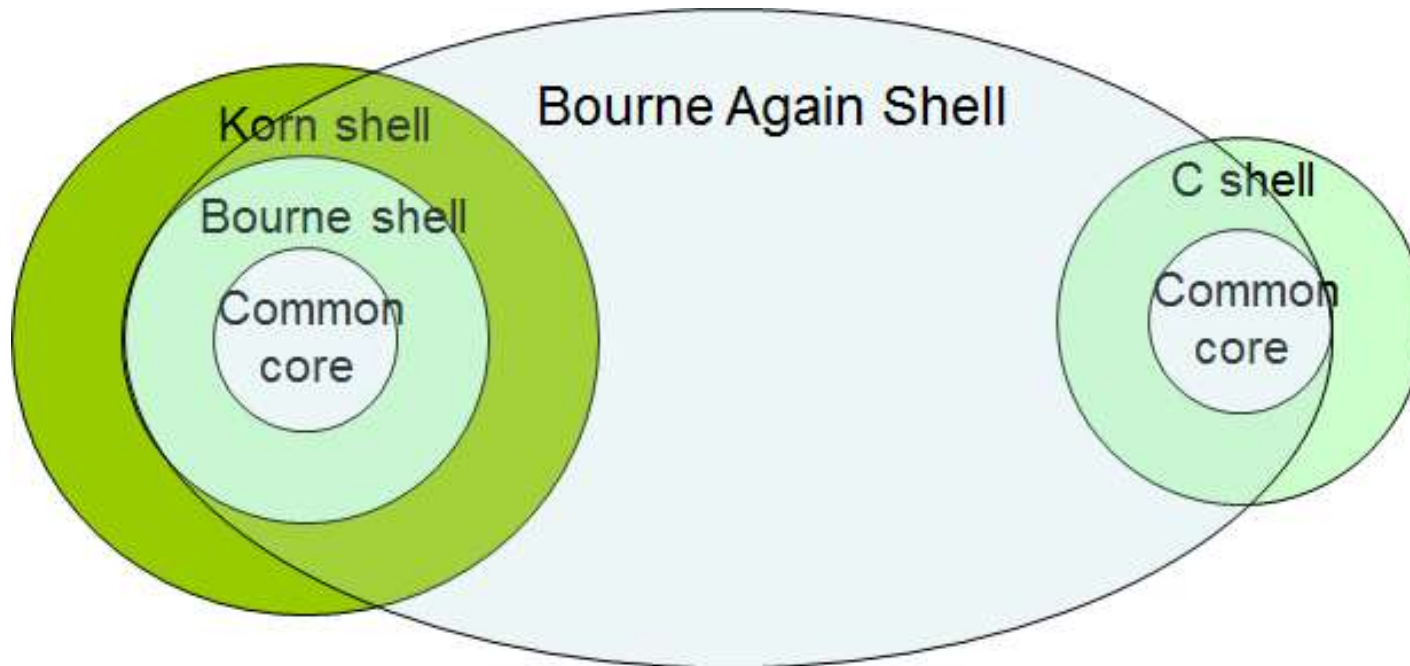
BASH

- BASH= Bourne Again Shell
- The shell's name is an acronym for Bourne Again Shell,
- **A pun on the name of the Bourne shell that it replaces and**
- **The notion of being "born again".**

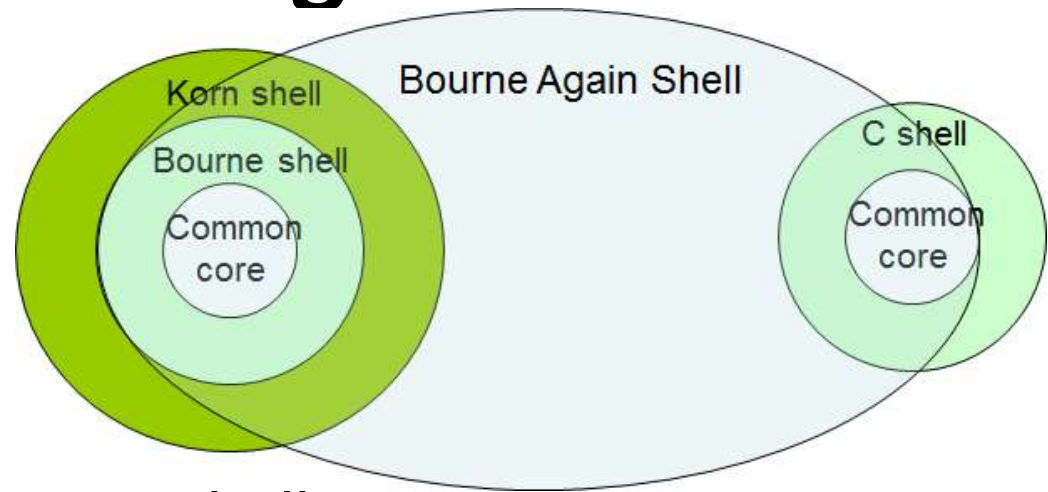
[https://en.wikipedia.org/wiki/Bash_\(Unix_shell\)#:~:text=Bash%20is%20a%20Unix%20shell,ported%20to%20Linux%2C%20alongside%20GCC.](https://en.wikipedia.org/wiki/Bash_(Unix_shell)#:~:text=Bash%20is%20a%20Unix%20shell,ported%20to%20Linux%2C%20alongside%20GCC.)

GNU Bourne-Again Shell

- The Bash command syntax is a superset of the Bourne shell command syntax.
- Bash command syntax includes ideas drawn from the KornShell (ksh) and the C shell (csh) such as command line editing, command history (history command), the directory stack, and POSIX command substitution syntax



GNU Bourne-Again Shell



- Denoted as **bash**
- It is compatible to the Bourne shell.
- It includes features from Korn and Bourne shell.
- It is most widely used shell in Linux systems.
- It is used as **default login shell in Linux systems and in macOS**. It can also be installed on Windows OS.

GNU ?

GNU ?

- **GNU is an operating system** and also an extensive collection of utility programmes wholly free software and also the project within which the free software concept originated
- **GNU is a recursive acronym for "GNU's Not Unix!"**
- **Chosen because GNU's design is Unix-like, but differs from Unix by being free software and containing no Unix code.**

Shell Scripting

- Usually shells are interactive that mean,
 - they accept command as input from users and execute them.
 - to execute a bunch of commands routinely,
 - **so we have to type in all commands each time in terminal.**

Shell Scripting

- As shell can also take commands
 - **as input from file we can write these commands in a file**
 - can execute them in shell to avoid this repetitive work.
 - These files are called **Shell Scripts or Shell Programs**.
 - Shell scripts are **similar to the batch file in MS-DOS**.
 - Each shell script is saved with **.sh file extension** eg.
myscript.sh

Shell Scripting

- A shell script have syntax just like any other programming language.
- A shell script comprises following elements –
 - **Shell Keywords – if, else, break etc.**
 - **Shell commands – cd, ls, echo, pwd, touch etc.**
 - **Functions**
 - **Control flow – if..then..else, case and shell loops etc.**

The Shell Definition Line

- The shell definition line tells the system
 - **what shell should be used** to interpret the script's commands, and
 - where the program (shell) is located.
- The shell definition line tells the system to use the **Korn Shell when executing this script.**
 - `(#!/bin/ksh)`

Shell Script Comments

- Comments, or non-command code,
- In a shell script, **Comments begin with the # (pound) character**

Permissions for script

- Before you can run the script, you need to make the shell script file, an executable by using the UNIX chmod command.

Permissions for script

- The following command will allow only the user (owner) of the script to execute it:

\$ chmod u+x script1

- If you wanted to allow everyone (all) to execute the script, you would use this command:

\$ chmod a+x script1

Executing script

- After the script file has been made an executable with the `chmod` command, you can run the script in a new shell by giving the path to the script:

\$./script1

- This `(./)` would be the path to **script1** if you are in the same **directory as the script**.
- If you were in a **different directory than the script**, you could use one of the following commands to run it:

\$ /home/student1/script1

or

\$ /bin/ksh /home/student1/script1

Executing script

- On some online portals, Execute it as
\$ sh script1.sh

Sample program

- `vi hello.sh`
- To go into edit mode:
 - **press ESC and type I**
- To save a file
 - **press ESC and type :w fileName**
- To save a file and quit:
 - **press ESC and type :wq**
 - Or
 - **press ESC and type :x**
- Save and close the file.
- You can run the script as follows:
./hello.sh
- Sample outputs:
bash: ./hello.sh: Permission denied
- Type **chmod u+x hello.sh**
- And execute the program again

Chmod command

Chmod command

The '*chmod command*' is used to change the access permissions of a file. The syntax is as follows:

chmod ugo+rwx filename Where,

- u : Users
- g : Groups
- o : Others
- + : Adds the permission
- - : Removes the permission
- = : Overwrites current permissions
- r : Read permission
- w : Write permission
- x : Execute permission

Displaying Output

- Two methods for displaying output to standard output:

echo "Text Line 1"

- The print command is the replacement for the echo command.
 - You can use the print command when you are writing shell scripts because
 - it is more powerful than the echo command, and
 - its syntax has been standardized on multiple operating systems.

Echo command

- **echo** - display a line of text.

Example-2:

To print value of x, where x=10.

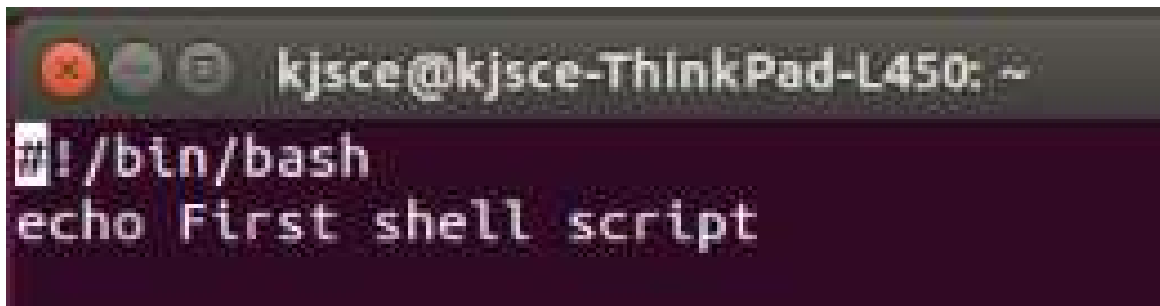
```
$ echo $x
```

output:

10

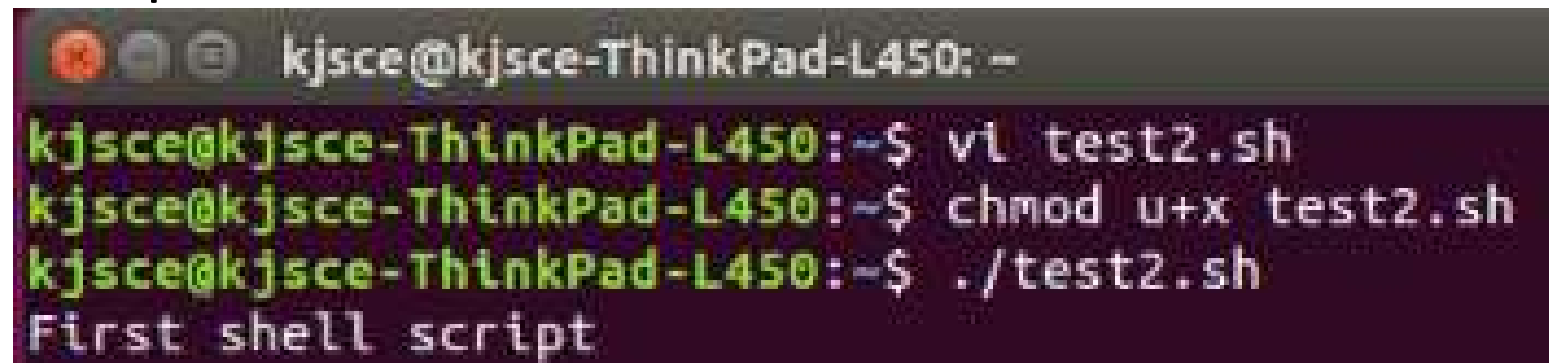
Echo command

- To display a line of text
- test2.sh-

A terminal window with a dark background. The title bar shows 'kjsce@kjsce-ThinkPad-L450: ~'. The prompt is '#!/bin/bash'. The command 'echo First shell script' is entered and executed, resulting in the output 'First shell script' on the next line.

```
kjsce@kjsce-ThinkPad-L450: ~  
#!/bin/bash  
echo First shell script
```

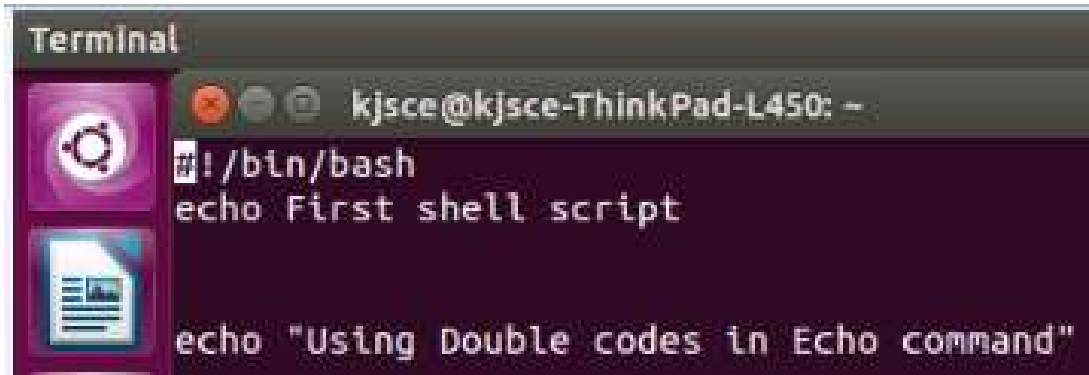
- Output-

A terminal window with a dark background. The title bar shows 'kjsce@kjsce-ThinkPad-L450: ~'. The prompt is 'kjsce@kjsce-ThinkPad-L450:~\$'. The commands 'vi test2.sh', 'chmod u+x test2.sh', and './test2.sh' are entered and executed in sequence. The output 'First shell script' is displayed on the last line.

```
kjsce@kjsce-ThinkPad-L450: ~  
kjsce@kjsce-ThinkPad-L450:~$ vi test2.sh  
kjsce@kjsce-ThinkPad-L450:~$ chmod u+x test2.sh  
kjsce@kjsce-ThinkPad-L450:~$ ./test2.sh  
First shell script
```

Echo command

- Using double quotes with echo
- test2.sh-



```
Terminal
kjsce@kjsce-ThinkPad-L450: ~
#!/bin/bash
echo First shell script
echo "Using Double codes in Echo command"
```

- Output-



```
kjsce@kjsce-ThinkPad-L450:~$ ./test2.sh
First shell script
Using Double codes in Echo command
```

Echo command

- **Using double quotes with echo**
- That the use of double quotes (") characters affect how spaces and TAB characters are treated, for example:

```
$ echo Hello      World
Hello World
$ echo "Hello      World"
Hello      World
```

Read command

- To read input from the user
- test2.sh-

A terminal window titled "Terminal" with a dark background. The window shows a shell script being executed. The script starts with a shebang line, followed by an echo statement. Then it uses the read command to get a username, followed by another echo statement. It then uses read to get three more usernames, followed by an echo statement that concatenates the input names with their respective variables.

```
Terminal
kjsce@kjsce-ThinkPad-L450: ~
#!/bin/bash
echo First shell script

echo "Using double quotes with echo"
echo "Enter username"
read user1
echo "The First User is $user1"
echo "Enter 3 other usernames"
read user2 user3 user4
echo "User2=$user2 , User3=$user3 and User4=$user4"
```

Read command

- To read input from the user
- test2.sh-



```
Terminal
kjsce@kjsce-ThinkPad-L450: ~
#!/bin/bash
echo First shell script

echo "Using double quotes with echo"
echo "Enter username"
read user1
echo "The First User is $user1"
echo "Enter 3 other usernames"
read user2 user3 user4
echo "User2=$user2 , User3=$user3 and User4=$user4"
```

- Output-

```
kjsce@kjsce-ThinkPad-L450:~$ ./test2.sh
First shell script
Using double quotes with echo
Enter username
SDC
The First User is SDC
Enter 3 other usernames
AAG JMR RAN
User2=AAG , User3=JMR and User4=RAN
kjsce@kjsce-ThinkPad-L450:~$
```


How to define variables

- We can define a variable by using the syntax `variable_name=value`.
- To get the value of the variable, add `$` before the variable.

Arithmetic expression

- Arithmetic expansion allows the evaluation of an arithmetic expression and the substitution of the result.
- The format for arithmetic expansion is:

`$((expression))`

- Reading man bash says that the old format:

`$(expression)`

- is deprecated and will be removed.
- Otherwise they should be equivalent.

Shell Script to Add Two Integers

```
#!/bin/bash
```

```
# Calculate the sum of two integers with pre initialize values
```

```
# in a shell script
```

```
a=10
```

```
b=20
```

```
sum=$(( $a + $b ))
```

```
echo $sum
```

Arithmetic Operators

Operator	Description	Example
+ (Addition)	Adds values on either side of the operator	`expr \$a + \$b` will give 30
- (Subtraction)	Subtracts right hand operand from left hand operand	`expr \$a - \$b` will give -10
* (Multiplication)	Multiplies values on either side of the operator	`expr \$a \* \$b` will give 200
/ (Division)	Divides left hand operand by right hand operand	`expr \$b / \$a` will give 2
% (Modulus)	Divides left hand operand by right hand operand and returns remainder	`expr \$b % \$a` will give 0
= (Assignment)	Assigns right operand in left operand	a = \$b would assign value of b into a
== (Equality)	Compares two numbers, if both are same then returns true.	[\$a == \$b] would return false.
!= (Not Equality)	Compares two numbers, if both are different then returns true.	[\$a != \$b] would return true.

Relational Operators

Operator	Description	Example
-eq	Checks if the value of two operands are equal or not; if yes, then the condition becomes true.	[\$a -eq \$b] is not true.
-ne	Checks if the value of two operands are equal or not; if values are not equal, then the condition becomes true.	[\$a -ne \$b] is true.
-gt	Checks if the value of left operand is greater than the value of right operand; if yes, then the condition becomes true.	[\$a -gt \$b] is not true.
-lt	Checks if the value of left operand is less than the value of right operand; if yes, then the condition becomes true.	[\$a -lt \$b] is true.
-ge	Checks if the value of left operand is greater than or equal to the value of right operand; if yes, then the condition becomes true.	[\$a -ge \$b] is not true.
-le	Checks if the value of left operand is less than or equal to the value of right operand; if yes, then the condition becomes true.	[\$a -le \$b] is true.

Shell Script –Using Decision constructs

Syntax

if [expression]

then

Statement(s) to be executed if expression is true

else

Statement(s) to be executed if expression is not true

fi

Conditional Expressions

- It is very important to understand that all the conditional expressions should be inside square braces with spaces around them,
- For example
- **[\$a == \$b]** is correct whereas,
- **[\$a==\$b]** is incorrect.

Conditional Expressions

- It is very important to understand that all the conditional expressions should be placed inside square braces with spaces around them.
- For example,
- `[ $a <= $b ]` is correct whereas,
- `[$a <= $b]` is incorrect.

Conditional Expressions

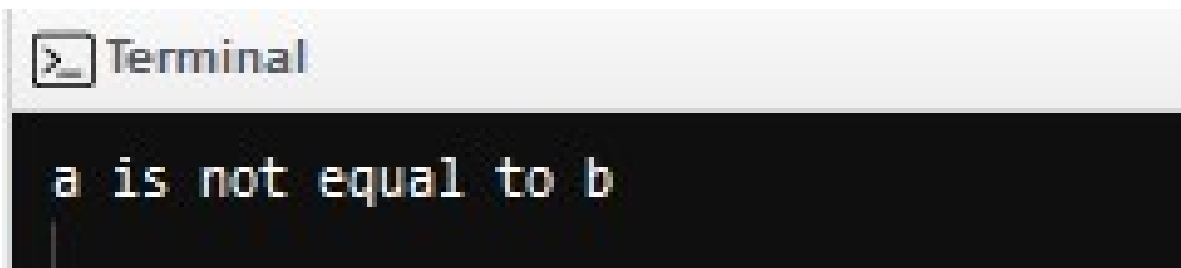
- -It is Less than which is used for condition checking
- < is used for Reading input from the files.
- The statement [\$num < 10] is just syntactic sugar for the command test \$num < 10, which means:
- Run the command test with one parameter (the content of the shell variable num), and
- associate the standard input of this invocation with a file named 10

Shell Script –Using Decision constructs

Example-

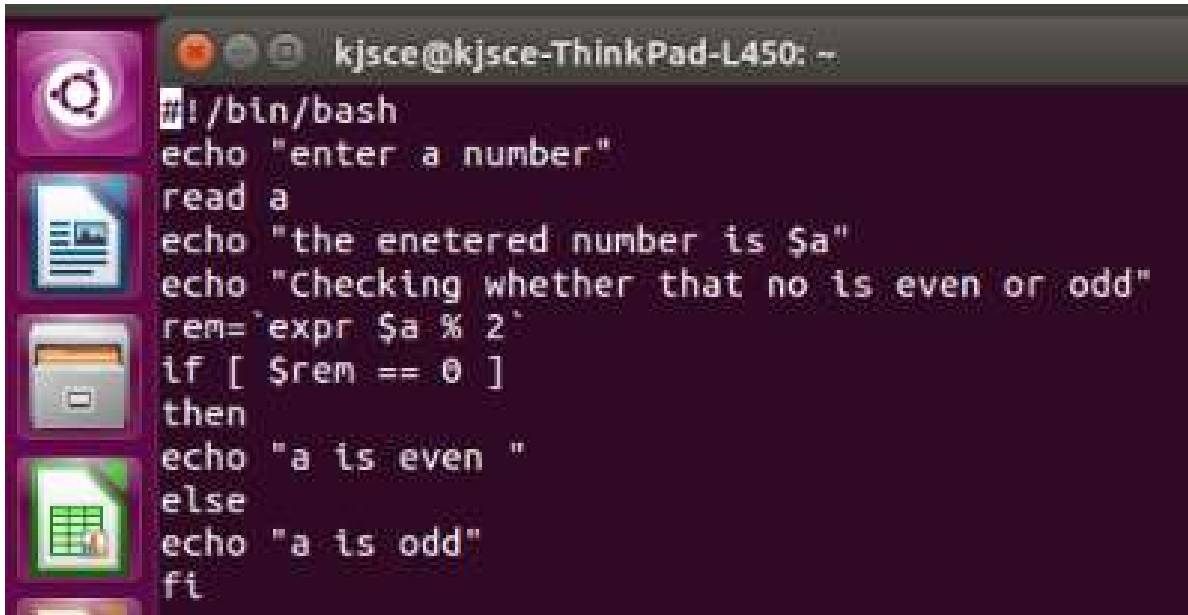
```
1  #!/bin/sh
2
3  a=10
4  b=20
5
6  if [ $a == $b ]
7  then
8      echo "a is equal to b"
9  else
10     echo "a is not equal to b"
11  fi
```

Output-



The image shows a terminal window with a title bar that says "Terminal". Inside the terminal, the output of the script is displayed as "a is not equal to b".

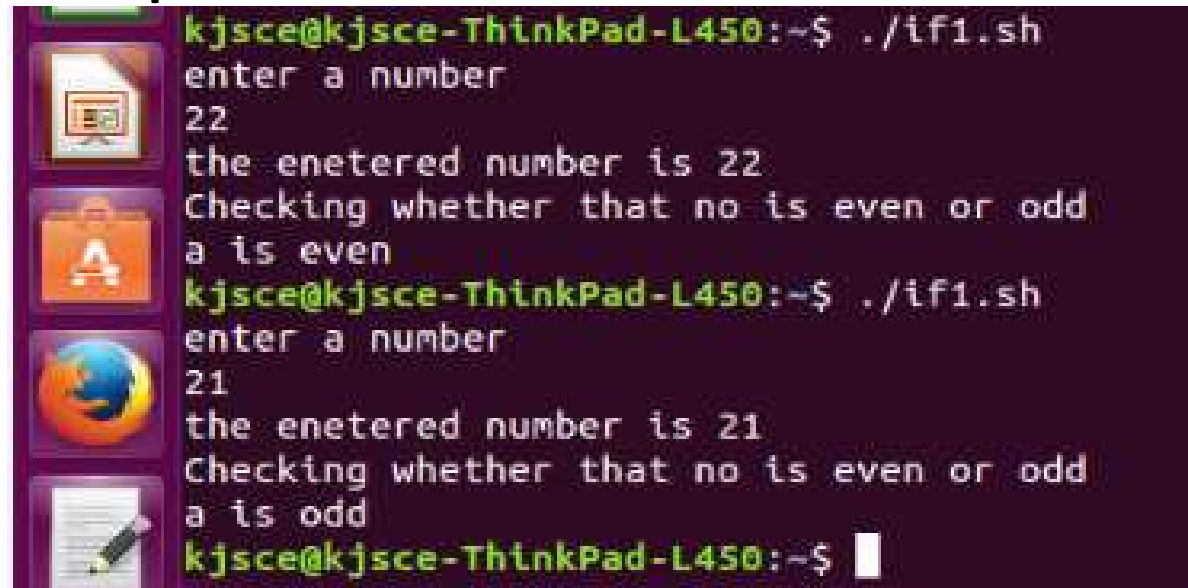
- **Shell Script to check if a number is even or odd**



```
kjsce@kjsce-ThinkPad-L450: ~  
#!/bin/bash  
echo "enter a number"  
read a  
echo "the enetered number is $a"  
echo "Checking whether that no is even or odd"  
rem=`expr $a % 2`  
if [ $rem == 0 ]  
then  
echo "a is even "  
else  
echo "a is odd"  
fi
```

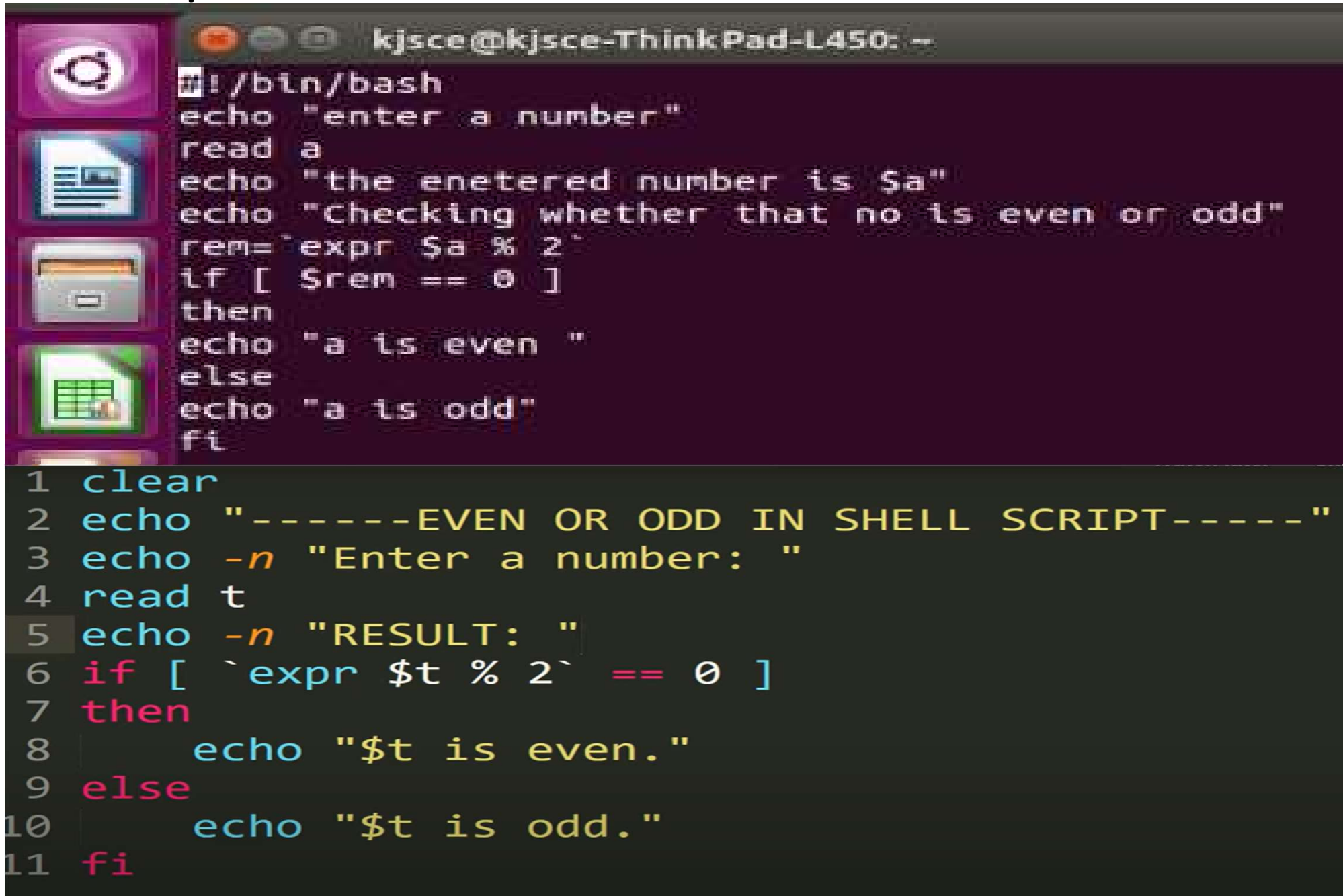
- *expr* is an external program used by Bourne shell.
- It uses *expr* external program with the help of backtick.
- The *backtick*(`) is actually called command substitution.

- **Output-**



```
kjsce@kjsce-ThinkPad-L450:~$ ./if1.sh  
enter a number  
22  
the enetered number is 22  
Checking whether that no is even or odd  
a is even  
kjsce@kjsce-ThinkPad-L450:~$ ./if1.sh  
enter a number  
21  
the enetered number is 21  
Checking whether that no is even or odd  
a is odd  
kjsce@kjsce-ThinkPad-L450:~$
```

- Shell Script to check if a number is even or odd



The image shows a terminal window with a dark purple background. The window title is 'kjsce@kjsce-ThinkPad-L450: ~'. The prompt is '# !/bin/bash'. The script content is as follows:

```
echo "enter a number"
read a
echo "the enetered number is $a"
echo "Checking whether that no is even or odd"
rem=`expr $a % 2`
if [ $rem == 0 ]
then
echo "a is even "
else
echo "a is odd"
fi
```

Below the terminal window, the same script is shown with line numbers and syntax highlighting on a dark background:

```
1 clear
2 echo "-----EVEN OR ODD IN SHELL SCRIPT-----"
3 echo -n "Enter a number: "
4 read t
5 echo -n "RESULT: "
6 if [ `expr $t % 2` == 0 ]
7 then
8     echo "$t is even."
9 else
10    echo "$t is odd."
11 fi
```

- **Shell Script to calculate hra and ta**

- Shell Script to calculate hra and ta

```
Desktop
kjsce@kjsce-ThinkPad-L450: ~
#!/bin/sh
echo "enter basic salary"
read basic
echo "if basic is less than 40,000 then hra=10% and ta=25% else hra=15% and ta=35%"
if [ $basic -lt 40000 ]
then hra=`expr 10 \* $basic / 100`
ta=`expr 25 \* $basic / 100`
echo "hra is $hra"
echo "ta is $ta"
else
hra=`expr 15 \* $basic / 100`
ta=`expr 35 \* $basic / 100`
echo "hra is $hra"
echo "ta is $ta"
fi
```

- *expr* is an external program used by Bourne shell.
- It uses *expr* external program with the help of backtick.
- The *backtick*(`) is actually called command substitution.

- Shell Script to calculate hra and ta

```
Desktop
kjsce@kjsce-ThinkPad-L450: ~
#!/bin/sh
echo "enter basic salary"
read basic
echo "if basic is less than 40,000 then hra=10% and ta=25% else hra=15% and ta=
35%"
if [ $basic -lt 40000 ]
then hra=`expr 10 \* $basic / 100`
ta=`expr 25 \* $basic / 100`
echo "hra is $hra"
echo "ta is $ta"
else
hra=`expr 15 \* $basic / 100`
ta=`expr 35 \* $basic / 100`
echo "hra is $hra"
echo "ta is $ta"
fi
```

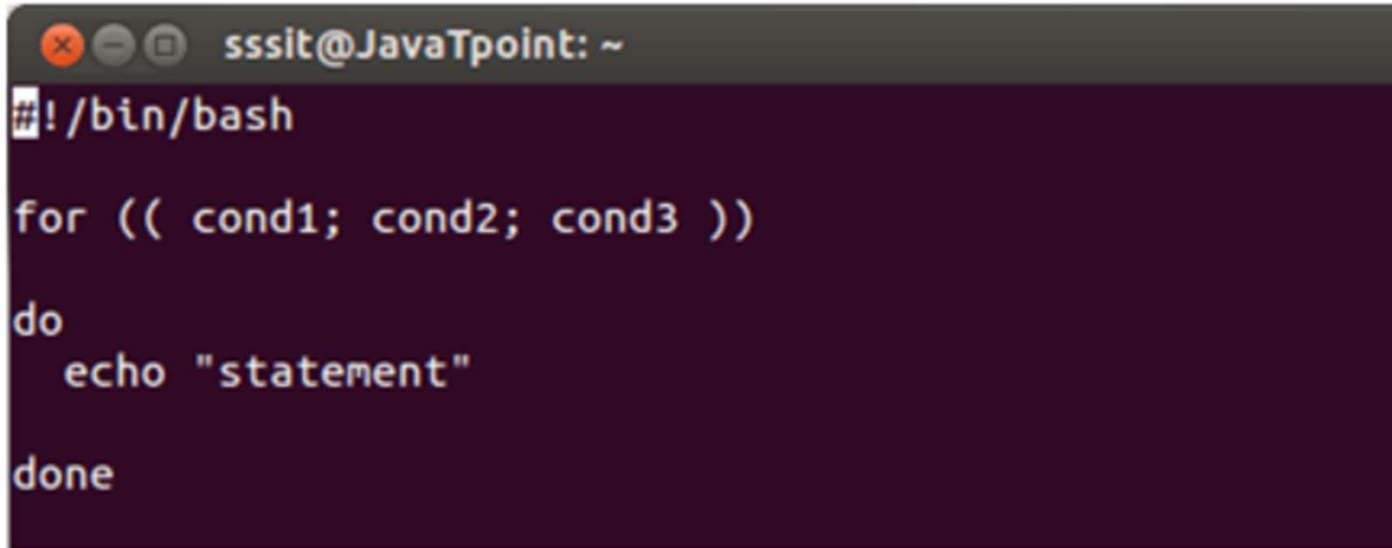
- Output-

```
kjsce@kjsce-ThinkPad-L450:~$ ./ifelse.sh
enter basic salary
50000
if basic is less than 40,000 then hra=10% and ta=25% else hra=15% and ta= 35%
hra is 7500
ta is 17500
kjsce@kjsce-ThinkPad-L450:~$ ./ifelse.sh
enter basic salary
35000
if basic is less than 40,000 then hra=10% and ta=25% else hra=15% and ta= 35%
hra is 3500
ta is 8750
```

- The backslash (\) character is used to mark these special characters so that they are not interpreted by the shell, but passed on to the command being run (for example, echo).

```
$ echo "A quote is \", backslash is \\, backtick is \`."
A quote is ", backslash is \, backtick is `.
```


For Loop Syntax

A terminal window with a dark purple background and light gray text. The window title bar shows 'sssit@JavaTpoint: ~'. The prompt is '#!/bin/bash'. The code displayed is a for loop: 'for ((cond1; cond2; cond3))', followed by an indented 'do' block containing 'echo "statement"', and ending with 'done'.

```
sssit@JavaTpoint: ~  
#!/bin/bash  
for (( cond1; cond2; cond3 ))  
do  
    echo "statement"  
done
```

condition1 indicates **initialization**,
cond2 indicates **condition** and cond3 indicates **update**.

For Loop Eg

```
sssit@JavaTpoint: ~  
#!/bin/bash  
  
for (( i=10; i >= 1; i-- ))  
do  
    echo "$i"  
done
```

O/P-

```
sssit@JavaTpoint: ~  
sssit@JavaTpoint:~$ ./rdom.sh  
10  
9  
8  
7  
6  
5  
4  
3  
2  
1  
sssit@JavaTpoint:~$
```